LAB ACTIVITY 3: APPLYING SOFTWARE PARADIGMS
Paradigm Calculator

Source Code: https://github.com/rhxhel/Arithmetic-Calculator.git

Group 4
Members:
Bernardino, John Renz
Matuto, Chelsy Rhaine
Paraiso, Maria Samantha
Sacatani, Juliana Mae
Revaula, Mark Daniel
Villafuerte, Angela Franceska

I. Learning Objectives
At the end of this activity, students should be able to design and implement the same problem using different software paradigms and explain how design approaches vary per paradigm.

II. System Description
The Paradigm Calculator is a versatile web-based application designed to perform arithmetic operations using multiple programming paradigms. The system is built to run in a web browser, handling user input from both on-screen buttons and keyboard entries, while preventing invalid operations such as division by zero.

III. Programming Languages Used
- **JavaScript –** Core language for implementing all calculator logic, including procedural, object-oriented, and functional paradigms.
- **HTML –** Structures the calculator interface, including buttons, input fields, and display areas.
- **CSS –** Styles the calculator, ensuring a user-friendly and visually appealing layout.

IV. Key Features

1. Arithmetic Operations – Performs standard arithmetic operations with error handling for invalid inputs:

- **Addition "+" –** Combines two numbers to give their total or sum.
- **Subtraction "-" –** Finds the difference by removing one number from another.
- **Multiplication "*" –** Repeated addition; calculates the product of two numbers.
- **Division "/" –** Splits a number into equal parts; gives the quotient of two numbers.
- **Modulus "%" –** Returns the remainder after dividing one number by another.

2. Multiple Paradigms – The calculator is implemented using three distinct programming approaches:

- **Procedural Paradigm:**
  Implements calculations through step-by-step procedures and functions. Input expressions are parsed and evaluated sequentially, making the flow straightforward and easy to follow.
- **Object-Oriented Paradigm (OOP):**
  Encapsulates calculator behavior within a **SmartCalculator** class. The class stores the current expression and provides methods to execute operations, promoting modularity and code reuse.
- **Functional Paradigm:**
  Uses pure functions to process expressions and return results without modifying external state. Each function is independent, enhancing predictability and testability.

3. Unit Converter – A tool that converts a value from one unit of measurement to another, such as meters to feet, kilograms to pounds, or Celsius to Fahrenheit, enabling easy comparison across different measurement systems.

4. User Input Handling

- Supports keyboard entry and on-screen buttons. Restricts other characters aside from numbers, operations, backspace (for delete), and enter (for result.)
- Keeps track of cursor position, allowing insertion or deletion at any point in the expression.
- After calculation, automatically manages subsequent input to either start a new calculation or continue using the last result.

5. Error Prevention

- Prevents operations that would result in undefined results, such as division by zero or invalid expression.

6. History Tracking

- Displays previous calculations for reference.

7. Interactive Web Interface

- Clean, responsive interface optimized for web browsers.
- Supports navigation via arrow keys for precise editing of expressions.

V. System Set-up

To run the Paradigm Calculator:

- Ensure Visual Studio Code is installed. Install the Live Server extension to enable real-time preview.
- Next, download or clone the source code from GitHub.
- Open the project folder in VS Code, then right-click the HTML file and select “Open with Live Server” to launch the calculator in a web browser.

This setup allows full interaction with the interface using both keyboard and on-screen buttons.

VI. System Workflow

1. Input – User enters an expression via keyboard or on-screen buttons in the web interface.

2. Chosen Paradigm – Users can select their preferred programming paradigm. They may also explore the script.js file and experiment by removing parts of the code:

- Without the OOP, Functional, or Procedural paradigms, the interface remains visible but calculations cannot execute.
- Without the Event-driven paradigm, the calculation logic exists but the buttons do not respond to user interaction.

3. Processing – The expression is evaluated according to the selected paradigm: procedural, OOP, or functional.

4. Validation – System checks for invalid operations (e.g., division by zero) before computing the result.

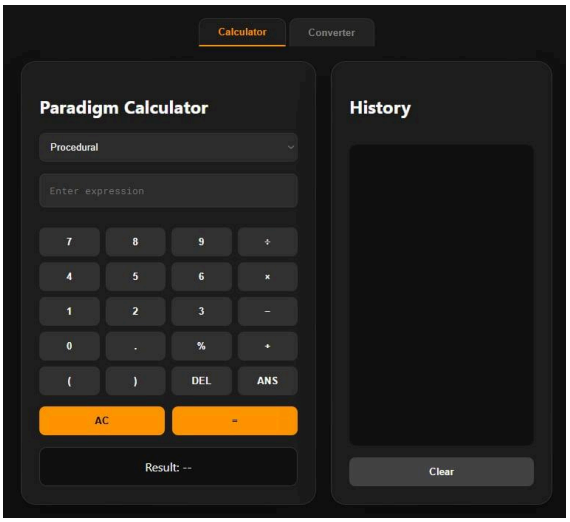
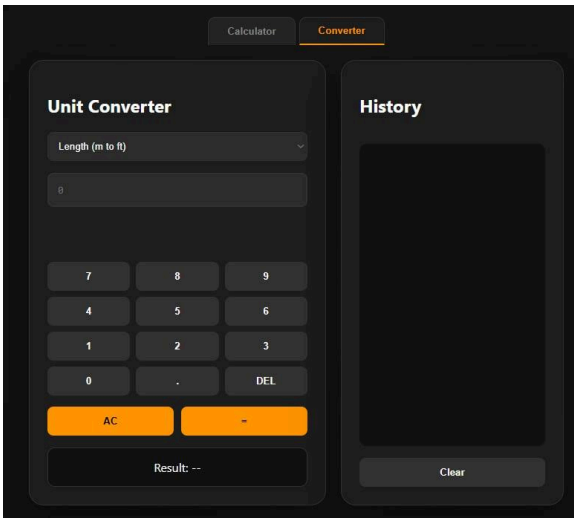
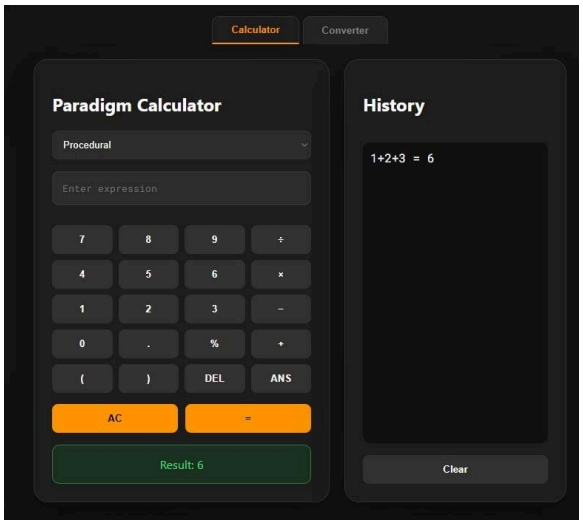
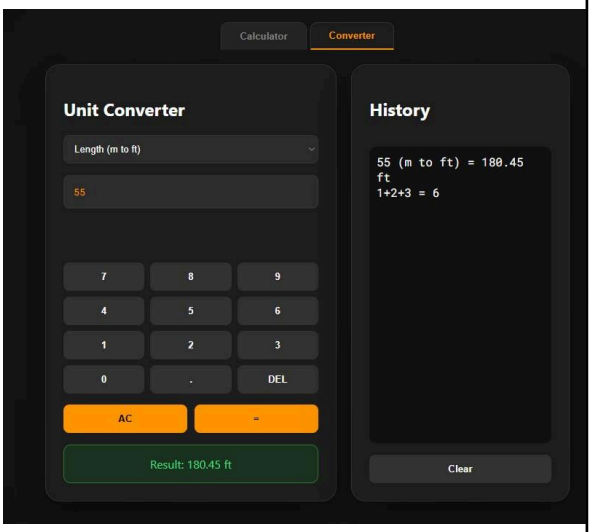
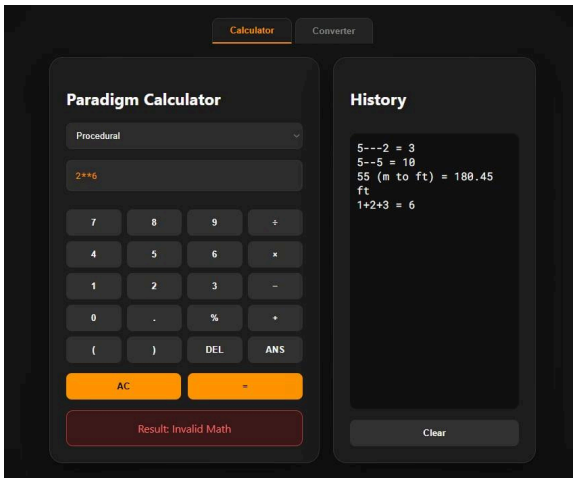
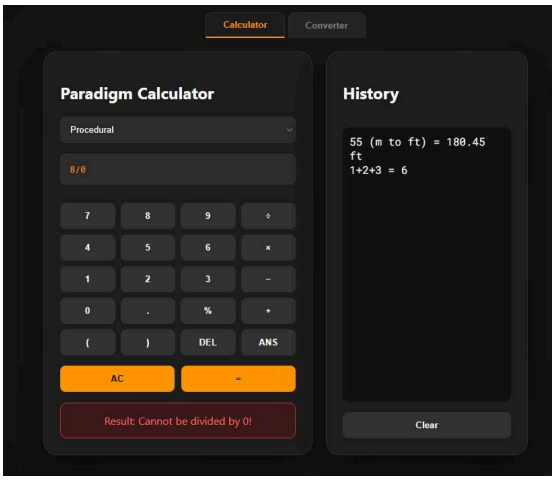
5. Output – Result is displayed in the input field, with history updated to show previous calculations.

6. Continuation – Users can either start a new calculation or append operators to the last result seamlessly.

VII. Benefits

- Flexible design demonstrating different programming paradigms for educational and practical purposes.
- Reliable input management ensures accurate and secure calculations.
- Web-based deployment allows accessibility across devices without installation.
- Modular code structure ma kes it easy to extend with additional features in the future.

VIII. Interface and Output

Interface	
Paradigm Calculator	**Unit Converter**
	

Sample Outputs